

Lab 5 Report

1 Overview

The following sections provide a high-level overview of each of the optimizations performed on the George Dantzig L4 compiler (commit hash: b924132d6e395aa0efd3591cf961d6ee82654522) at both the O0 (minimal and local) and O1 optimization levels.

Quantitative analysis was run on a Samsung Flex 2 Alpha in performance mode with an 11th-gen Intel i7. Specifically, analysis was taken from running the following commands:

```
../timecompiler --autograde | tail -n 1 > res.txt
python3 ../score_table.py res.txt
```

A major addition to the compiler was the `--unsafe` flag, which forgoes any memory access checks that C0 does. By removing these checks, large numbers of branches are removed from the code, making it faster at the expense of memory safety. Figure 1 shows the performance of the benchmark test cases when run with and without the `--unsafe` flag.

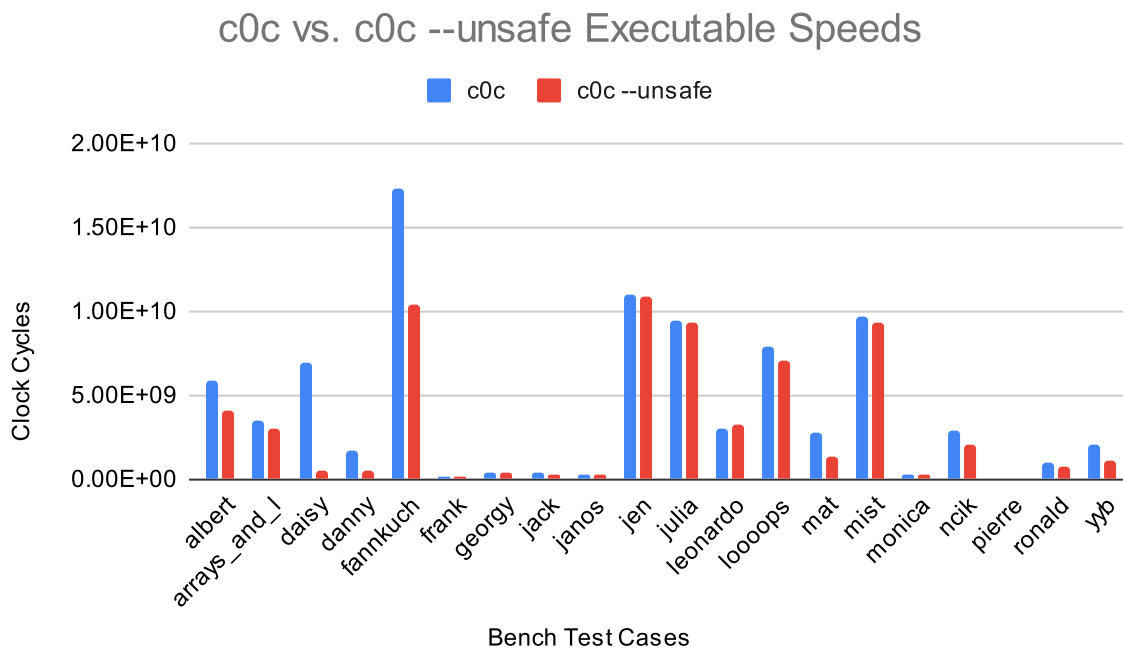


Figure 1: Number of clock cycles taken for each bench test case, run both with and without the `--unsafe` flag. Unsafe code almost always resulted in faster code.

2 Optimizations at O0

2.1 Peephole

Peephole optimizations were done using a rolling window after SSA, prior to register allocation. If an instruction was removed or simplified, the window would stay in place and simplifications would continue to be applied until no more changes could be made. Additionally, local peephole optimizations were done after register allocation immediately prior to output to remove self-moves.

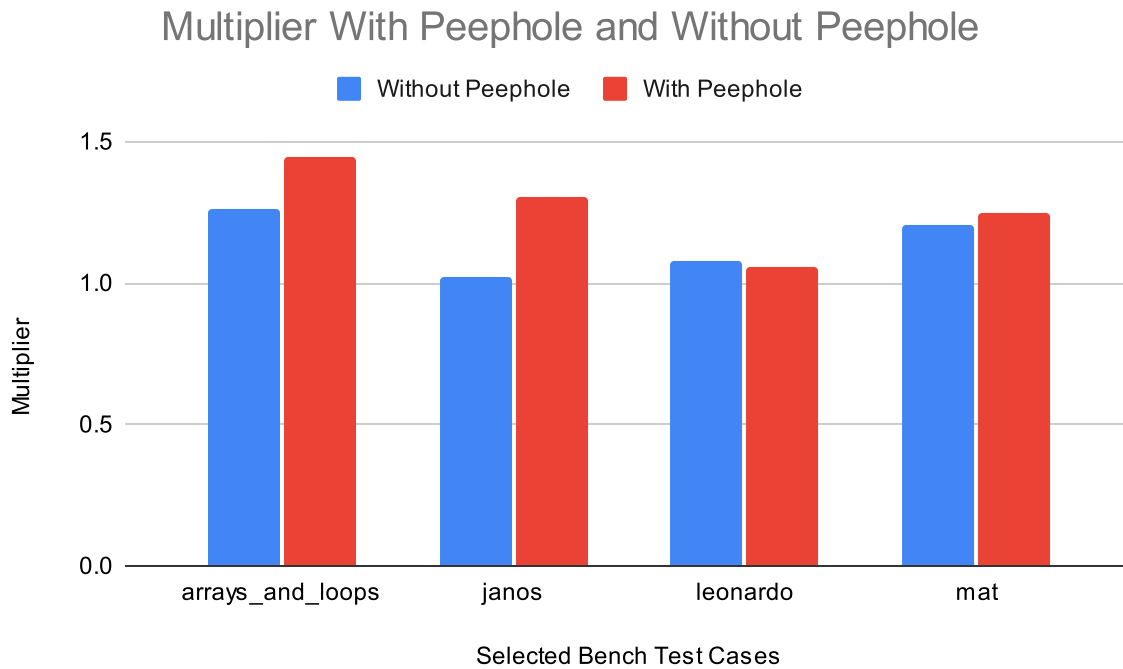


Figure 2: The results of including and removing peephole optimizations.

2.1.1 Strength Reduction

The following basic arithmetic strength reductions were performed on the AST:

$$\begin{array}{lcl}
 x = a + 0 & \longrightarrow & x = a \quad | \quad x = a - 0 \longrightarrow x = a \\
 x = a * 1 & \longrightarrow & x = a \quad | \quad x = a * 0 \longrightarrow x = 0
 \end{array}$$

2.1.2 Null Sequence Removal

As described in the lecture notes on peephole optimizations, the following null sequences were eliminated:

2.1.3 Stacked labels	2.1.4 Stacked gotos	2.1.5 Redundant moves	2.1.6 Self moves
.l1 .l1			$x \leftarrow x \longrightarrow \text{nop}$
.l2 $\longrightarrow$	goto l1 $\longrightarrow$.l1	$x \leftarrow y \longrightarrow x \leftarrow y$	
.l3	.l1	$y \leftarrow x$	

Figure 2 shows the result of peephole optimizations on

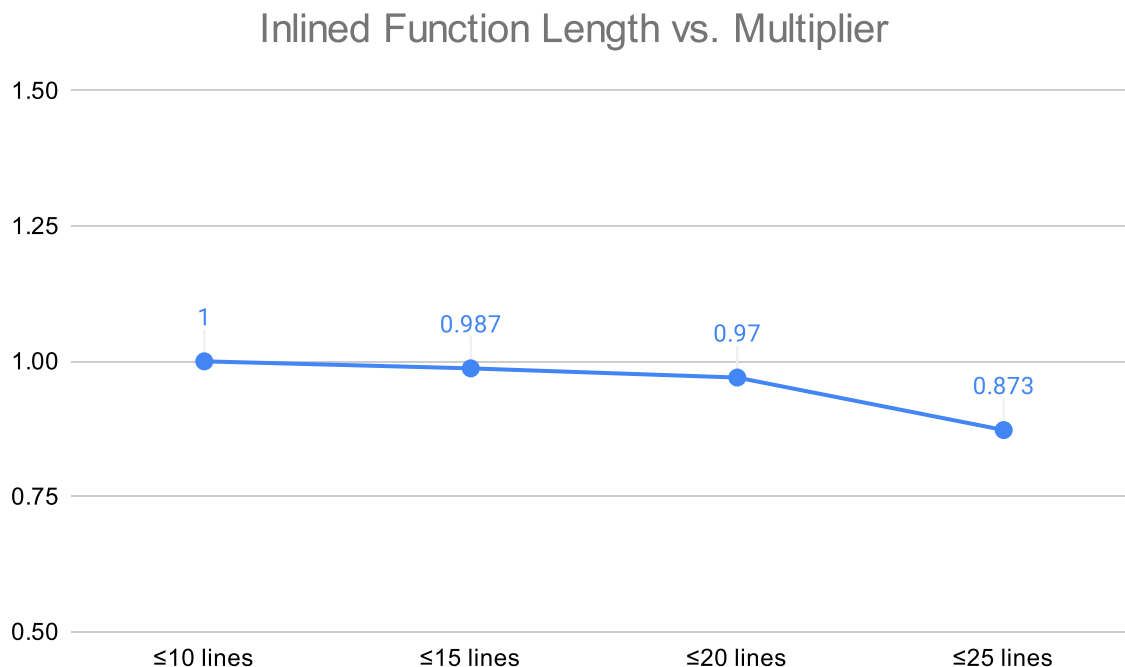


Figure 3: Average multiplier with comparison to `gcc` for benchmark tests when inlined functions have length at most 10 through at most 25. There is a clear downwards trend as the number of lines in the function grows.

3 Optimizations at O1

3.1 Function Inlining

Function inlining was performed on the IR, with the primary goal being to reduce unnecessary stack space use and allow for inter-function propagation. To inline, functions instructions were looked up and directly concatenated with the existing function's accumulation of instructions up to the function call. The inlining directive was made to be compatible with tail recursive calls - allowing for direct returns from inlined functions.

After AST to IR translation, the new IR instructions were iterated over, and every call statement triggered an inlining attempt. To avoid infinite inline nesting of recursive definitions, a map of previously inlined functions was kept during iteration, and passed down nested function inlines. Any new inlining attempts to functions f were first checked against the map to ensure f had not previously been inlined in the sequence of inlines to the current inlined function.

To avoid code bloat, functions longer than a preset length (10 instructions) were not inlined. Additionally, functions were only inlined to a maximum length of 15,000 lines to avoid timeouts.

Figure 3 shows the results of inlining functions of length at most 25, at most 20, at most 15, and at most 10.

The maximum number of instructions required to inline a function appears to have an inverse relationship to the average speedup. Since the instruction length is at the level of the AST, this is not particularly surprising. A function of length 10 in the AST representation can expand to be much larger in an IR. Since the compiler has a limit to how long a function can get, shrinking the size of the functions to inline can allow for more functions to be inlined, since larger functions will take up more of this limit.

3.2 Tail Call Optimization

Tail call optimization is rooted in the AST, which distinguishes between recursive tail calls and external function tail calls. The latter was supported only for functions requiring only register arguments (fewer than six arguments) to facilitate the elimination of the caller function's stack space prior to the tail call. New AST fields were introduced to propagate these statements down to the IR, where general tail calls were elaborated into `gotos` to the header of the function. On the other hand, recursive tail calls jumped directly to the body of the function - after parameter load in, and instead directly assigned arguments to local function variables:

3.2.1 Recursive tail calls

<pre>fun rec (x1, x2) = x1 <- rdi x2 <- rsi .bodycommon sub ... rec (x1', x2')</pre>	→	<pre>fun rec (x1, x2) = x1 <- rdi x2 <- rsi .body ... x1<-x1' x2<-x2' goto .body</pre>
--	---	--

3.2.2 Recursive Accumulators

Recursive calls were pattern matched for associative operations - specifically addition and multiplication. If all recursive calls in a function matched this associative operation, then the function was marked as an accumulative tail call-recursive function. For these functions, a new temporary variable, t' , was introduced. On external calls, t' was assigned 1 for multiplicative accumulation functions and 0 for additive accumulation functions at the start of the function. Returns were elaborated to include the binary operation with t' .

3.2.3 Recursive accumulation calls

<pre>fun rec (x1, x2) = x1 <- rdi x2 <- rsi .body if(...) return x1 ... return 4 + rec (x1', x2')</pre>	→	<pre>fun rec (x1, x2) = x1 <- rdi x2 <- rsi t' <- 0 .body if(...) return t'+x1 ... x1<-x1' x2<-x2' t'<-t'+4 goto .body</pre>
---	---	--

3.3 Dead Code Elimination

Dead code elimination is performed both before and during SSA, replacing unneeded instructions with `nop` instructions. Dead code elimination prior to SSA is performed following with the neededness analysis described in the lecture notes. Neededness analysis is stored along with each line, and propagated in `dce.ml`.

Dead code elimination performed during SSA comes in two forms: removal of empty basic blocks ("dead block elimination") and removal of instructions defining unneeded temps ("dead temp elimination").

Dead block elimination is performed by removing all blocks composed entirely of `nop` instructions, and blocks which cannot be reached by the function entry point:

1. Basic blocks were iterated over using a depth-first-search of the control-flow-graph of each function, starting from the function's entry point.

2. Each basic block's instructions were iterated and nop instructions were eliminated.

2.1 If a basic block consisted of entirely nop instructions except for the final "block transition" instruction (e.g. goto, return, jumpif, etc), the block was considered as a candidate for a "dead" block.

3. If a dead block candidate had a single successor, the block was marked as dead and its predecessors were directly connected to this successor. Otherwise, a dead block with multiple successors must end with a conditional jump, which was matched to the form:

$$\text{jumpif}\{e; l1; l2\}$$

3.1 If $l1=l2$ or e was a constant, this was elaborated into a goto, and the block was marked as dead.

4. If a block was marked as dead in the previous step, its predecessors were checked as dead block candidates, since their "block transition" functions had changed. This facilitated iterative "loop munching" that eliminated unnecessary loops and conditional jumps.

Dead temp elimination is performed using the following steps:

1. Create a directed graph where each vertex represents a temp, and edges represent dependencies for the definition of a temp. For example, the instruction:

$$t_1 \leftarrow t_2 \oplus t_3$$

would result in the edges (t_1, t_2) and (t_1, t_3) being added to the graph.

2. Mark effectful instructions, temps related to the return result, and control-flow instructions as *mandatory*, and propagate this status to every vertex reachable from the initial mandatory set. This is implemented as a depth-first search starting from the mandatory set.
3. Remove instructions defining temps that are not needed.

3.4 Constant Propagation and Folding

Constant propagation and constant folding are performed simultaneously on each basic block during SSA. In particular, constant propagation and folding are performed on each basic block, and the results are propagated to blocks they dominate. A global map of temps to their associated constants is stored and updated as constant folding is performed on each basic block.

In order to propagate constants between basic blocks, the blocks are iterated over in *reverse-postorder*. This ensures that temps that have constant values will be found before their uses.

Quantitatively, the addition of constant propagation and constant folding resulted in a significant increase in performance. However, constant propagation requires dead code elimination for its effects to be seen. 4.1 discusses the improvements further.

3.5 Common Subexpression Elimination

Similarly to constant propagation, common subexpression elimination is performed simultaneously on each basic block during SSA. Unlike constant propagation, CSE is not propagated to other basic blocks due to a lack of time for debugging. This limits its effectiveness significantly, especially when it could be propagated inside a loop. However, because it is linear time for each basic block, CSE is inexpensive to perform.

Figure 4 shows the results of performing CSE on the bench test cases compared to not performing CSE.

As expected, there is not much of a difference between using and not using CSE. Only one case showed a meaningful improvement in using CSE, *janos*. This may be the result of repetitive array accesses resulting in common subexpressions.

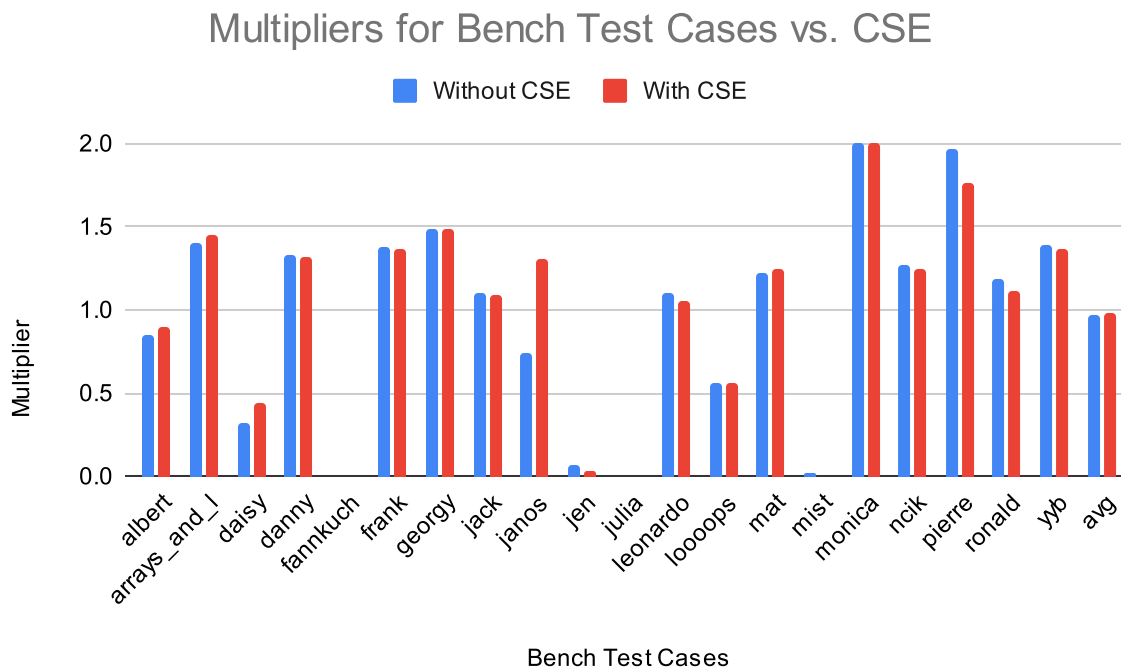


Figure 4: Average multiplier with comparison to gcc for benchmark tests when using CSE and when not using CSE. There is not much of a difference between using and not using CSE.

3.6 MCS Coloring Heuristic

To improve the efficacy of register allocation, we added a tie-breaking heuristic to our MCS algorithm to favor coloring deeply nested temps first. Specifically, after building a loop-nest tree (modeled after the one specified in the Appel textbook), the tree is traversed to determine which blocks are deeply nested in loops. From there, temps are numbered according to how deep in a loop they occur: if a temp is numbered with the deepest loop it is either *used* or *defined* in.

This priority is then used in the MCS algorithm to determine which temps to color first, with the aim of making it more likely to assign temps deep inside loops to registers. Since these temps are more deeply nested, they are likely used the most often.

3.6.1 Loop Hoisting

Loop hoisting was implemented according to the lecture 19 notes, with invariant instructions being propagated throughout the loop-nest tree. Hoisting uses the following steps:

1. Insert preheader nodes before every loop, identifying loops with the loop-nest tree. Preheader nodes are initially empty and immediately precede the header node of each loop.
2. Generate an ordering of loops such that the deepest loops are hoisted first using BFS.
3. For every loop, propagate invariance according to the rules from lecture. In particular, any expression composed of invariant expressions which is *not* defined by a phi function or used in a phi function is marked as invariant. Using this information, hoist instructions defining a variable with only invariant computation out of the loop to the preheader.

We had minimal improvement on adding loop hoisting since it requires SSA to be pruned; otherwise, unnecessary phi functions mean that any instruction inside a loop will be defined by a useless phi function.

3.7 Register Coalescing

To minimize the moves introduced when exiting out of SSA, and independently minimize redundant moves outside of the copy propagation capable of being performed only in SSA, greedy register coalescing was performed. The algorithm used was as described in the register optimization lecture:

1. After coloring, and during color-register assignment, the colored pseudo-assembly instructions were iterated. Any move $x \leftarrow y$ triggered an attempt to coalesce registers.
2. If x and y interfered, the iteration continued. Otherwise, the union of the neighborhoods of x and y were searched to find the minimum absent color c' .
3. If c' was less than 13, the number of available registers, then x and y were assigned c' , and the coloring graph was updated. Otherwise, the iteration continued.

3.8 Control Flow Optimization

The control flow graph was used to order assembly basic blocks in a way that optimized fall-through. Specifically, blocks were ordered in their depth-first-search ordering, greedily creating a maximal pattern of stacked `gotos` as seen in 2.1.4, which were then sub-sequentially matched by the peephole optimizations.

4 Optimization Interactions

4.1 Dead Code Elimination and Constant Propagation

General performance improved when dead code elimination and constant propagation were combined. Specifically, performing dead temp elimination prior to and following constant propagation had the greatest impact on performance.

Figure 5 shows the impact of combining dead code elimination with constant propagation. Removing the dead code elimination pass prior to constant propagation had a small negative impact on performance, but the average speedup only decreased from 1.081 to 1.044. However, removing the dead code elimination pass following constant propagation resulted in a significant penalty to performance, with most test cases timing out and an average multiplier of 0.194.

These results are to be expected, since introducing SSA form creates many new temps. If dead code elimination is not performed during SSA form, the result will be many new instructions added to the code, hindering performance. Constant propagation does not change the number of instructions, so without dead code elimination the amount of time taken will be roughly the same, regardless of whether constant propagation is done. Dead code elimination allows temps that are now unused and not necessary to be removed, reducing the number of instructions.

We note in this figure that some cases, like `monica` or `georgy`, did not show much improvement when constant propagation and dead code elimination were applied. These cases included many memory accesses, which our constant propagation does not propagate through, so it was minimally helpful.

On the other hand, cases such as `pierre` had huge improvements in speed. This is likely due to a *combination* of constant propagation and function inlining. Inlining the recursive calls in `pierre` allows constant propagation to be performed across multiple calls.

5 Weaknesses

5.1 Alias Analysis

Because this compiler does not have any alias analysis, `load` and `store` operations immediately stop optimizations. Programs that will work poorly with this compiler are therefore ones that do many loads from and stores into the same memory location; these would be caught by alias analysis as able to be optimized so as to not recompute the same address multiple times. For example, `mist` from the bench cases includes the following loop:

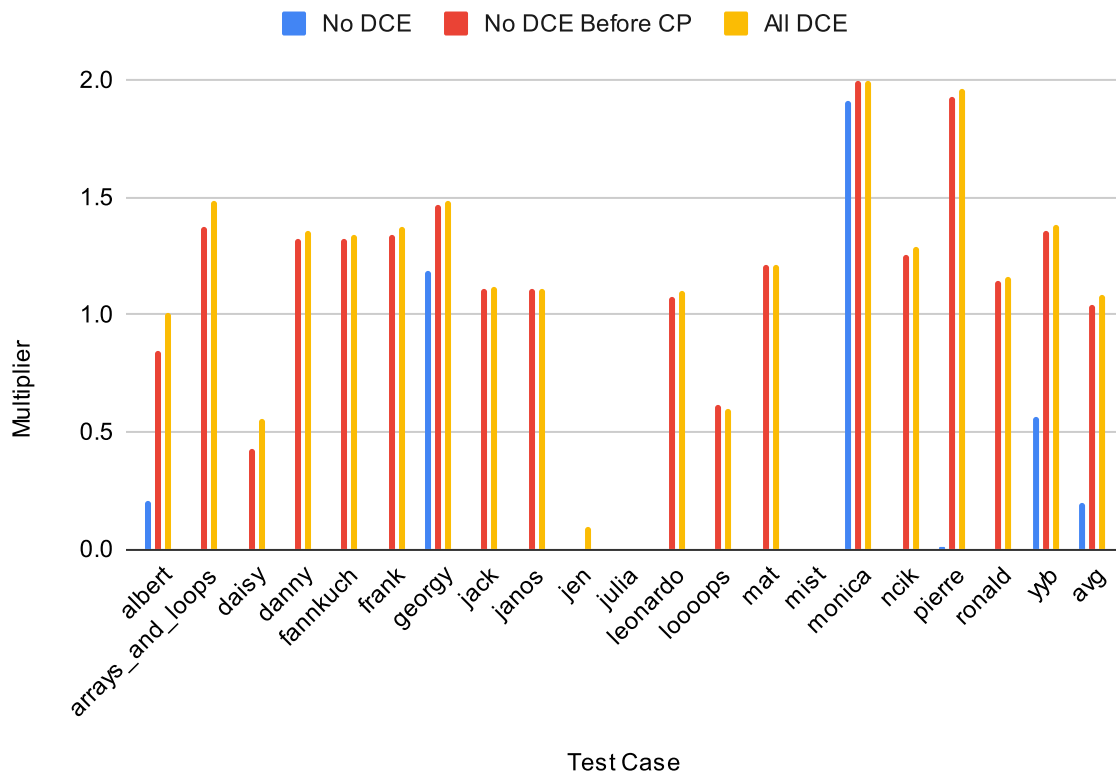


Figure 5: Comparison of different placements of dead code elimination during SSA, before and after constant propagation. Cases where dead code elimination was performed after constant propagation had significant speedups when compared to the case where no dead code elimination was performed. Run with MCS coloring and CSE disabled.


```
for (int i = 0; i < n; i++) {
    sum += a[i] * a[i];
}
```

Here, `a[i]` is accessed twice, but the lack of alias analysis means that the compiler will do a bounds and null check both times. This adds a large amount of unnecessary branching. Similarly, the compiler could use alias analysis to only access this memory location once, and compute the multiplication using the temp, but instead will pull from memory twice.

5.2 Loop Optimizations

Another weakness in this compiler is the lack of (working) loop hoisting and induction variable identification, as well as other more advanced loop optimizations. While constant propagation and constant subexpression elimination can help in removing unnecessary computation, many loop-invariant computations are performed during every iteration of the loop. The code from `jen` is a good example of this. The code has several nested loops, and with more advanced loop optimizations such as induction variables, branches could be predicted during compilation. These loop optimizations would also make our compiler perform better on recursive code, since our function inlining creates loops that could then be optimized.

5.3 Adversarial Code

Our compiler is therefore particularly weak to code which uses many loops and loads and stores from memory in those loops. One example of code like this is `greenhaskellcurry-pest0` from the L4 test cases, which employs a large loop and does many memory accesses inside the loop. A better compiler might use loop unrolling, since the number of iterations is inferrable at compile-time, and alias analysis to notice that `L[i + 1]` is used twice.

```
void mixer(int[] L, int numElems) {
    for (int i = 0; i < numElems - 1; i++) {
        int temp = L[i];
        L[i] = L[i + 1] - 2;
        L[i + 1] = temp + 1;
    }
}

int main() {
    int numElems = 1000;
    int[] L = alloc_array(int, numElems);

    L[0] = 199547;
    ... (omitted)
    L[999] = 187631;

    int iterations = 0;
    while (true) {
        mixer(L, numElems);
        for (int i = 0; i < numElems; i++) {
            if (L[i] >= 0) {
                if (L[i] == 0) {
                    assert(iterations == 174719);
                }
                L[1 / L[i]];
            }
        }
    }
}
```

```
        iterations += 1;
    }
    return iterations;
}
```